FIAP

PROMPT ENGINEERING & ARTIFICIAL INTELLIGENCE

Checkpoint 02
Prompt Toolkit

Projeto Python em Grupo · Módulo 2 · Aulas 05 a 08

🐍 Projeto Python (não Colab)

👥 Grupo até 5 pessoas

📦 Repositório + README + PDF

⏰ 1 semana após Aula 08

01

O Projeto

Construir um toolkit Python que aplica automaticamente as 4 técnicas de prompting a qualquer tarefa de negócio, compara resultados e recomenda a melhor abordagem.

Visão Geral

O grupo constrói um **projeto Python organizado em módulos** (não um notebook) que funciona como uma ferramenta de linha de comando ou script. O toolkit recebe uma tarefa de domínio, aplica Zero-Shot, Few-Shot, CoT e Role Prompting, mede qualidade e custo de cada abordagem e gera um relatório comparativo.

O resultado final é uma **aplicação que qualquer desenvolvedor pode usar** para encontrar o melhor prompt para qualquer tarefa — basta definir o domínio, os inputs e os exemplos.

📦 **ENTREGA**

1. **Repositório** com a estrutura de pastas descrita abaixo (pode ser ZIP ou GitHub)
2. **README.md** — Como instalar, configurar e executar o projeto
3. **PDF** — Documentação do projeto: domínio, arquitetura, resultados, guia de bolso, reflexão

Nomeação: `CP02_[NomeDoGrupo].zip` + `CP02_[NomeDoGrupo].pdf`

⚙️ **STACK OBRIGATÓRIA**

Linguagem: Python 3.10+
LLM: Ollama API (local/gratuito) com modelo `gpt-oss:120b`
Tokens: `tiktoken` para contagem
Visualização: `matplotlib` + `pandas`
Ambiente: `venv` ou `pip install -r requirements.txt`
⚠️ **SEM API paga** — tudo gratuito

02

Arquitetura do Projeto

Estrutura de pastas e módulos que o grupo deve seguir.

Estrutura de Pastas Obrigatória

Arquitetura do Repositório

```
prompt-toolkit/
├── README.md                  # Como instalar e rodar
├── requirements.txt           # Dependências do projeto
├── .env.example               # Exemplo de variáveis de ambiente
├── main.py                    # Ponto de entrada — executa o toolkit
├── src/
│   ├── __init__.py
│   ├── llm_client.py          # Conexão com Ollama API (Aula 05)
│   ├── prompt_builder.py      # Montar prompts por anatomia (Aula 05)
│   ├── techniques.py          # 4 técnicas: ZS, FS, CoT, Role (Aulas
06+07)
│   ├── tasks.py               # Tarefas do domínio: classif, extração...
(Aula 08)
│   ├── evaluator.py           # Medir qualidade, tokens, consistência
│   └── report.py              # Gerar tabelas e gráficos comparativos
├── data/
│   ├── inputs.json            # 5+ inputs reais por tarefa
│   └── examples.json          # Exemplos para few-shot
├── prompts/
│   ├── system_prompts.json    # Personas para role prompting (Aula 07)
│   └── templates.json         # Templates de prompt por tarefa
├── output/
│   ├── resultados.csv         # Resultados da execução
│   └── graficos/              # PNGs dos gráficos gerados
└── docs/
    └── CP02_NomeDoGrupo.pdf  # Documentação do projeto
```

Fluxo de Execução

Quando o usuário roda `python main.py`, o sistema deve:

Fluxo: main.py

```
# 1. Carregar configurações (.env, data/inputs.json, prompts/)
# 2. Para cada TAREFA definida em tasks.py:
#    a. Carregar 5 inputs reais do data/inputs.json
#    b. Aplicar 4 técnicas (ZS, FS, CoT, Role) via techniques.py
#    c. Para cada técnica × cada input:
#       - Montar prompt via prompt_builder.py
#       - Enviar ao LLM via llm_client.py
#       - Medir tokens + tempo via evaluator.py
#    d. Comparar resultados com output esperado
# 3. Gerar relatório via report.py:
#    - Tabela comparativa (CSV + terminal)
#    - Gráficos (PNG salvos em output/graficos/)
#    - Recomendação da melhor técnica por tarefa
# 4. Executar teste de temperatura (0.1, 0.5, 1.0) no melhor prompt
```

💡 **DIAGRAMA DO FLUXO**

`inputs.json` → `prompt_builder` → `techniques (ZS/FS/CoT/Role)` → `llm_client` → `evaluator` → `report` → `output/`

03

Detalhamento dos Módulos

O que cada arquivo .py deve conter e qual aula referencia.

Módulo: `llm_client.py` + `prompt_builder.py` (Aula 05)

`llm_client.py`

- Classe `LLMClient` com método `chat(prompt, system, temp, max_tokens)`
- Usar `requests` para chamar a API REST do Ollama (`http://localhost:11434/api/chat`)
- Host do Ollama via variável de ambiente (`.env`)
- Tratamento de erros (timeout, rate limit, retry)
- Retornar dict: `{"resposta", "tokens_prompt", "tokens_resposta", "tempo_ms"}`

`prompt_builder.py`

- Função `montar_prompt(instrucao, contexto, input_dados, formato_output)`
- Separar instrução de dados (princípio da Aula 05)
- Validar que nenhum componente está vazio
- Função `adicionar_exemplos(prompt, exemplos)` para few-shot
- Função `adicionar_cot(prompt, passos)` para chain-of-thought

Módulo: `techniques.py` (Aulas 06 + 07)

Implementar 4 funções, uma por técnica. Cada função recebe **tarefa + input** e retorna o **prompt montado**:

FUNÇÃO	O QUE FAZ	REF.
`zero_shot(tarefa, input)`	Monta prompt direto usando `prompt_builder`. Sem exemplos. Instrução clara + formato de output definido.	Aula 06
`few_shot(tarefa, input, exemplos)`	Monta prompt com 2-3 exemplos do `data/examples.json`. Formato: `Input: "..." → Output: "..."`.	Aula 06
`chain_of_thought(tarefa, input, passos)`	Monta prompt com instrução de raciocínio explícito: "Analise passo a passo: 1... 2... 3...".	Aula 06
`role_prompting(tarefa, input, persona)`	Usa system prompt com persona do `prompts/system_prompts.json`. Retorna tupla (system, user).	Aula 07

🎭 **SOBRE AS PERSONAS (AULA 07)**

O arquivo `prompts/system_prompts.json` deve conter pelo menos **2 personas** detalhadas para o domínio do grupo. Cada persona deve incluir: experiência, especialidade, tom de voz e limitações. Não basta "Você é um assistente" — detalhe conforme Aula 07.

Módulo: `tasks.py` (Aula 08)

Definir **no mínimo 3 tarefas** do domínio escolhido. Cada tarefa é um dict com:

Estrutura de uma tarefa

```
# Cada tarefa em tasks.py deve ter:
tarefa = {
    "nome": "classificacao_sentimento",
    "tipo": "classificacao",     #
classificacao|extracao|sumarizacao|geracao|traducao|code
    "instrucao": "Classifique como POSITIVO, NEGATIVO, NEUTRO ou MISTO",
    "formato_output": "Responda APENAS com a classificacao",
    "exemplos_fewshot": [      # Para few-shot
        {"input": "Adorei!", "output": "POSITIVO"},
        {"input": "Pessimo.", "output": "NEGATIVO"},
    ],
    "passos_cot": [            # Para CoT
        "Identifique aspectos positivos",
        "Identifique aspectos negativos",
        "Compare e classifique",
    ],
    "persona": "analista_cx",  # Chave do system_prompts.json
}
```

Tipos de tarefa obrigatórios (escolha 3+ do domínio):

Classificação

Sentimento, urgencia, categoria, intent, prioridade

Extração

NER, dados estruturados (JSON), entidades, valores

Sumarização

Resumo executivo, bullet points, abstract

Geração

Descrições, respostas, emails, FAQ, código

Módulo: `evaluator.py` + `report.py`

`evaluator.py`

- `contar_tokens(texto)` — via tiktoken
- `medir_acuracia(resposta, esperado)` — match exato ou por keywords
- `medir_consistencia(respostas[])` — mesma pergunta N vezes, % iguais
- `testar_temperatura(prompt, temps[])` — rodar com 0.1, 0.5, 1.0
- Cada resultado salvo como dict para o report

`report.py`

- `gerar_tabela(resultados)` — DataFrame com pandas, salvar CSV
- `grafico_acuracia(resultados)` — barras agrupadas por técnica
- `grafico_custo(resultados)` — tokens médios por técnica
- `grafico_temperatura(resultados)` — consistência por temp
- `recomendar(resultados)` — melhor técnica por tarefa + justificativa

Dados: `data/inputs.json`

Cada tarefa deve ter **no mínimo 5 inputs reais** com o output esperado:

data/inputs.json (exemplo para e-commerce)

```
{
  "classificacao_sentimento": [
    {"input": "Produto excelente, chegou rapido!", "esperado": "POSITIVO"},
    {"input": "Veio quebrado, pessimo.", "esperado": "NEGATIVO"},
    {"input": "Bom preco mas qualidade media.", "esperado": "MISTO"},
    // ... mais 2 inputs
  ],
  "extracao_dados": [
    {"input": "Notebook Dell de R$3500 com pixels mortos",
     "esperado": {"produto": "Notebook Dell", "preco": "R$3500", "defeito":
"pixels mortos"}},
    // ... mais 4 inputs
  ]
}
```

⚠️ **INPUTS REAIS**

Os inputs devem ser **textos reais ou realistas** do domínio (reviews verdadeiras, trechos de documentos, mensagens de clientes). Não invente frases genéricas — pesquise exemplos reais.

Documentação (PDF — 4 a 6 páginas)

1. Capa: nome do grupo, membros (RM), domínio, nome do toolkit
2. Domínio e problema: por que este domínio? Qual problema real o toolkit resolve?
3. Arquitetura: diagrama visual do fluxo (inputs → techniques → evaluator → report). Pode ser feito no draw.io, Mermaid ou à mão.
4. Stack técnica: lista de bibliotecas, modelo gpt-oss:120b via Ollama, versão do Python, como executar
5. Resultados: tabela comparativa + gráficos gerados pelo sistema (screenshots ou exports)
6. Guia de bolso: tabela "quando usar cada técnica" para o domínio
7. Reflexão: o que surpreendeu? Qual técnica venceu? O que fariam diferente?

Rubrica de Avaliação

CRITÉRIO	PESO	EXCELENTE (10)	BOM (7)	INSUFICIENTE (4)
Arquitetura e organização	**15%**	Estrutura de pastas correta, módulos separados, requirements.txt, .env.example, README funcional	Estrutura parcial, falta README ou .env	Tudo em 1 arquivo ou sem organização
4 técnicas implementadas Aulas 06+07	**25%**	ZS + FS + CoT + Role funcionais, reutilizáveis, com personas detalhadas	3 técnicas ou personas genéricas	Menos de 3 ou não funciona
3+ tarefas do domínio Aula 08	**20%**	3+ tipos (classificação, extração, geração...) com 5 inputs reais cada	2 tipos ou inputs genéricos	1 tipo ou sem inputs
Análise e visualização	**20%**	Tabela CSV + 3 gráficos (acurácia, custo, temperatura) + recomendação automática	Tabela e 1 gráfico	Sem visualização
Documentação (README + PDF)	**20%**	README com instruções claras, PDF com diagrama de arquitetura e guia de bolso	README básico, PDF sem diagrama	Sem README ou sem PDF

📅 **ENTREGA**

Prazo: 1 semana após Aula 08 · **Grupo até 5** · `CP02_[NomeDoGrupo].zip` + `CP02_[NomeDoGrupo].pdf`

FIAP

Checkpoint 02 — Prompt Toolkit

🐍 Projeto Python modular (não notebook)

🔧 4 técnicas: ZS + FS + CoT + Role × 3 tarefas × 5 inputs

📊 Relatório automático: tabelas + gráficos + recomendação

📄 README.md + PDF com arquitetura e guia de bolso

Copyright © 2026 Prof. Jorge Luiz Gomes · FIAP